SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Mukesh Raj L	**Reg. No.:**	192571036
Date:		**Duration:**	60 minutes

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.

6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified

Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollbac k details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/docum entation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

CI/CD PIPELINE DESIGN

AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform

1. Project Requirement and CI/CD Analysis

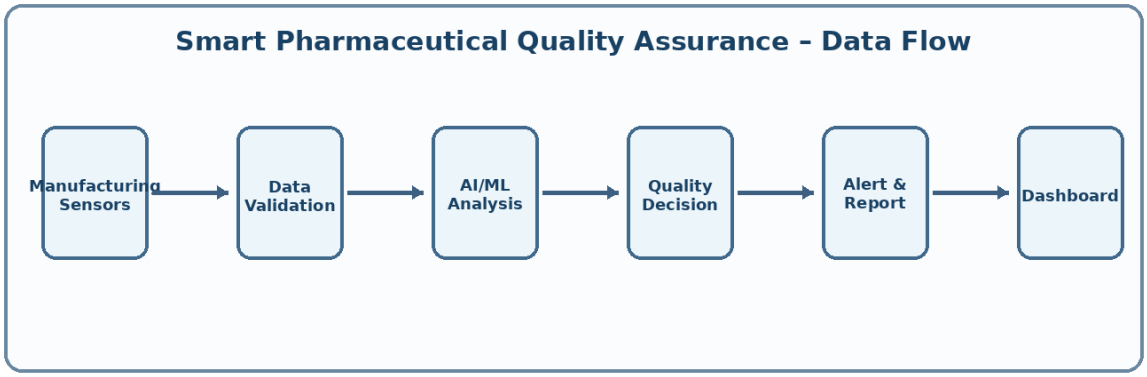


Figure: High-level data and quality-assurance flow

Functional reliability is especially important because the platform processes manufacturing-related quality information. The pipeline must therefore verify not only that the application starts successfully, but also that important quality workflows continue to produce valid responses after every software change.

The project can contain multiple components such as a user interface, backend APIs, database services, AI/ML analysis modules, alerting functions, and reporting services. CI/CD must coordinate changes across these components and provide clear evidence of which version of each component was tested and deployed.

Traceability is another key requirement. Each release should be associated with a source commit, build identifier, automated test result, security result, image version, deployment record, and rollback history. This creates a clear audit trail for the software delivery lifecycle.

The proposed platform applies Artificial Intelligence and data-driven quality assurance to pharmaceutical manufacturing. It collects manufacturing and quality data, analyses batch conditions, detects anomalies, predicts possible quality deviations, and generates quality reports and alerts. Because the platform supports a quality-critical manufacturing process, every software change must be tested, traceable, secure, and deployable with minimal downtime.

The CI/CD pipeline is designed to automatically integrate code changes, build the application, execute multiple levels of testing, analyse code quality, scan for security issues, package approved releases, deploy them through controlled environments, monitor the deployed system, and support rapid rollback.

- Frequent and controlled integration of application, AI/ML, API, and database changes.
- Automated build and dependency management to produce reproducible releases.
- Unit, integration, API, regression, and user-interface testing.
- Validation of AI service interfaces and quality-rule workflows before deployment.
- Static code analysis, dependency vulnerability scanning, and secret detection.
- Container-based packaging for consistent execution across development, staging, and production.
- Quality gates that block deployment when tests fail or critical issues are detected.
- Traceable versioning, audit logs, notifications, monitoring, and rollback support.

2. Proposed CI/CD Pipeline Architecture and Workflow

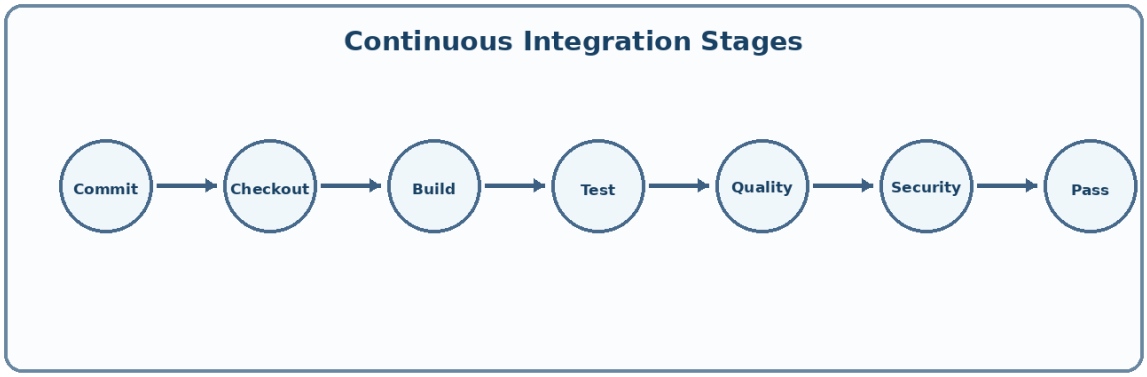
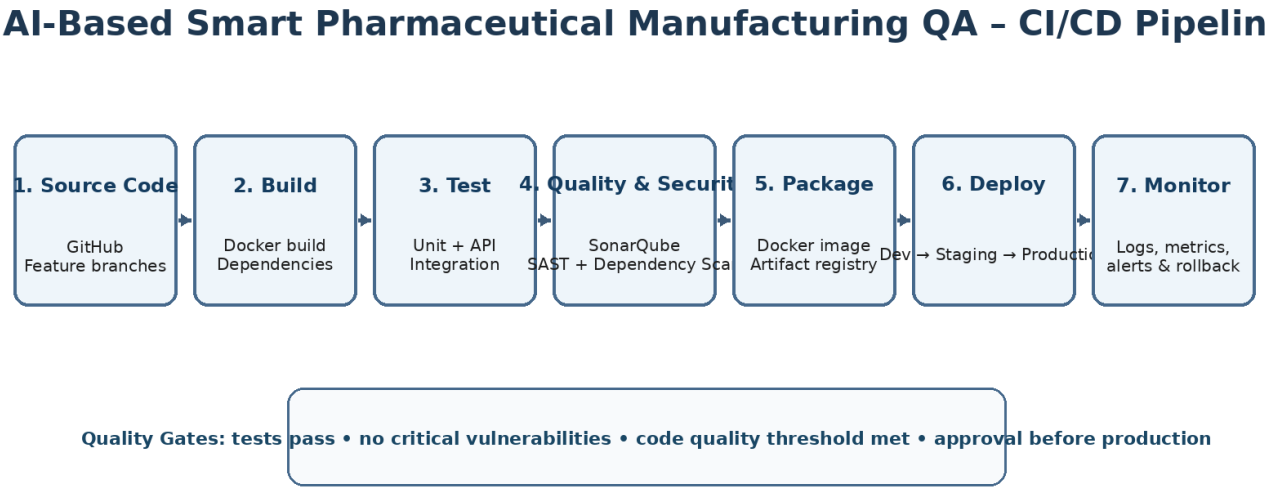


Figure: Continuous integration stages

The workflow is designed as a sequence of controlled gates. A later stage cannot start until the previous stage completes successfully. For example, packaging is not performed when unit tests fail, and Production deployment is not allowed when a security scan identifies a critical issue.

The pipeline can run automatically on commits and pull requests. This gives developers early feedback and reduces the possibility of accumulating untested changes. Important branches can also require successful checks before a merge is accepted.



Workflow: A developer commits code to a feature branch or main branch in GitHub. GitHub Actions automatically starts the pipeline. The system is built and tested, followed by code-quality and security checks. If all quality gates pass, a versioned Docker image is created and stored in a container registry. The release is deployed first to the Development environment, then to Staging for broader validation, and finally to Production after approval. Monitoring continuously checks application health and quality-service behaviour. If a production release becomes unstable, the previous stable version is restored.

3. Tools and Technologies with Justification

The selected tools are intended to work together rather than operate as isolated utilities. GitHub provides source control and workflow triggers, GitHub Actions coordinates automation, testing tools validate behaviour, SonarQube evaluates code quality, security tools analyse risks, Docker creates a portable release artifact, and monitoring tools provide feedback after deployment.

The exact tools may be replaced by equivalent enterprise tools without changing the overall pipeline design. The essential requirement is that each pipeline stage has an automated, repeatable, and traceable mechanism.

Pipeline Stage	Tool / Technology	Justification
Source Code Management	GitHub	Version control, branching, pull requests, reviews, and change traceability.
CI Orchestration	GitHub Actions	Direct repository integration and automated workflows on commits or pull requests.
Build & Packaging	Docker	Creates reproducible, versioned application images.
Automated Testing	PyTest / Jest / Postman-Newman	Supports unit, integration, API, regression, and workflow testing.
Code Quality	SonarQube	Detects bugs, code smells, duplication, and maintainability issues.
Security Scanning	Trivy + secret scanning	Detects vulnerable dependencies, container issues, and exposed secrets.
Image Registry	GitHub Container Registry	Stores approved versioned container images.
Deployment	Docker Compose / Kubernetes	Provides consistent and scalable deployment.

4. Automated Testing and Quality Integration

Test cases should include both expected and unexpected conditions. For example, batch input may contain missing fields, out-of-range values, duplicate identifiers, delayed sensor information, or malformed API requests. Automated tests should verify that the system handles such situations safely and produces understandable errors.

Where AI/ML services are involved, the software interface should also be tested for model-service availability, correct request formatting, valid response parsing, timeout handling, and safe fallback behaviour. Model changes should be versioned so that a deployed prediction service can be linked to the correct application release.

- Unit Testing: tests individual functions such as data validation, batch scoring, anomaly detection interfaces, authentication, and report generation.
- Integration Testing: verifies communication among the web application, backend APIs, AI/ML service, database, and notification module.
- API Testing: validates request/response behaviour, authentication, error handling, and critical REST APIs.
- Regression Testing: re-runs the main functional suite after significant changes to ensure existing features are not broken.
- Data Validation Tests: checks mandatory fields, valid ranges, formats, missing values, and incorrect sensor or batch inputs.
- Security Testing: runs dependency and container scans and checks for exposed secrets.
- Code Quality Gate: SonarQube checks maintainability, duplication, bugs, and configured quality thresholds before packaging.
- Smoke Testing After Deployment: confirms core services, login, AI quality analysis, database connectivity, and alert generation.

A pipeline run is considered successful only when the required tests pass and the configured code-quality and security gates are satisfied. Failed gates immediately stop the next deployment stage and notify the team.

5. Deployment Strategy

Deployment configuration should be treated as controlled infrastructure. Environment-specific values such as database endpoints, API keys, and monitoring settings must be managed securely and separately from application source code.

Before promotion, the pipeline should verify that the release artifact is immutable. The same tested image should move from Development to Staging and then to Production instead of rebuilding the application separately for each environment.

Environment	Purpose	Deployment Rule
Development	Rapid integration and developer verification.	Automatically deploy approved main-branch builds after CI checks.
Testing / Staging	Production-like validation of complete workflows.	Deploy after Development success; run integration, regression, API, and smoke tests.
Production	Live operation of the pharmaceutical QA platform.	Deploy only after all gates pass and release approval is provided.

The deployment uses versioned Docker images so the exact application version can be identified and reproduced. Staging uses the same release artifact that is promoted to Production, reducing environment-related differences.

6. Security, Monitoring, Notifications and Rollback

Security checks should use severity thresholds. Informational findings may be reported without stopping the pipeline, while critical findings can block release promotion until they are reviewed and resolved.

Monitoring should distinguish between normal operational variation and meaningful failures. Health endpoints, error-rate thresholds, response-time trends, service availability, and failed quality-processing requests are useful indicators for determining whether a release remains healthy.

- Secrets such as database credentials and API keys are stored securely and are never hard-coded in source code.
- Pull requests and code reviews provide controlled changes to important branches.
- Static analysis and vulnerability scanning are executed before a release is approved.
- Container images are scanned before deployment, and only approved versions are promoted.
- Application logs, pipeline logs, deployment versions, and important quality events are retained for traceability.
- Prometheus collects health and performance metrics, while Grafana dashboards show service status and anomalies.
- Pipeline failures and production alerts trigger notifications to the development and QA team.
- Rollback uses the previously known stable container version; failed health checks can trigger restoration of the previous release.

7. Pharmaceutical QA Platform Overview

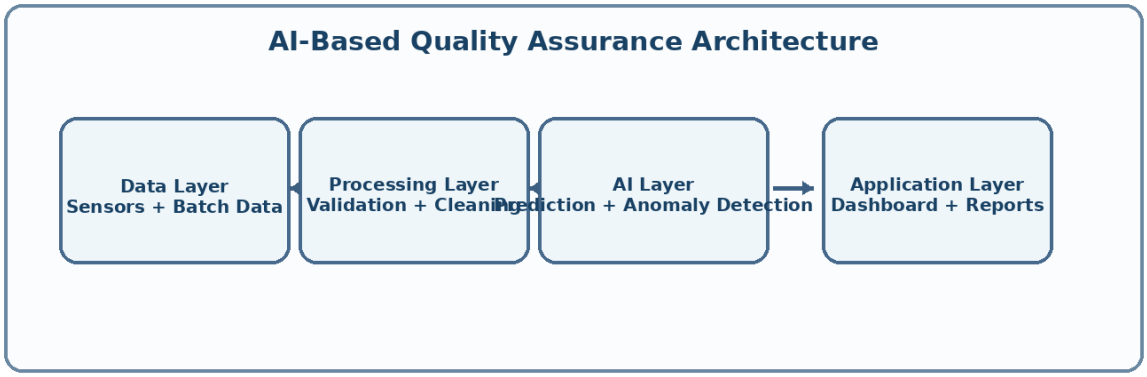
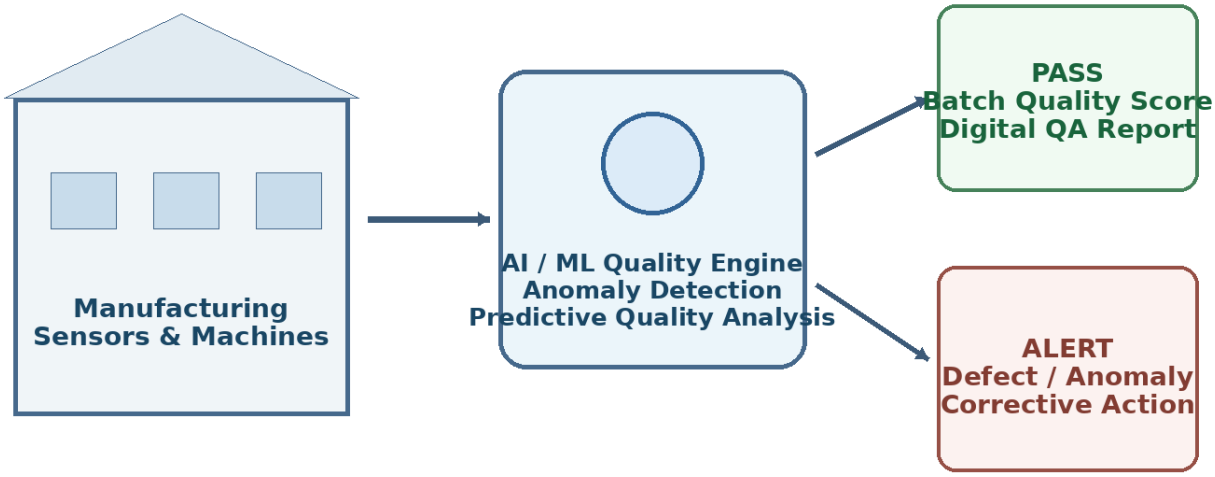


Figure: Layered architecture of the smart QA platform

The platform can provide a single view of quality-related software outputs by combining incoming process information, AI-assisted analysis, rule-based checks, and digital reporting. The objective of the CI/CD process is to protect the software platform that performs these tasks; it does not replace physical pharmaceutical validation procedures.

Alerts generated by the platform should be traceable to the relevant input, processing event, and software version. This improves the ability to investigate why an alert was generated and which deployed version handled the data.



The platform connects manufacturing data sources with an AI/ML quality engine. Incoming measurements and batch information are validated and analysed to produce quality scores, anomaly alerts, and digital quality reports. The CI/CD pipeline protects this software workflow by ensuring that only tested, secure, and approved versions reach each deployment environment.

8. CI/CD Workflow from Code Commit to Deployment

The workflow also supports feedback to developers. A failed stage returns logs and test information so that the responsible change can be corrected. Once the correction is committed, the pipeline executes again using the same defined sequence.

Release approvals should be based on visible pipeline evidence rather than informal assumptions. Test results, quality-gate status, security findings, deployment checks, and monitoring readiness provide the information required for a controlled decision.

1. Developer pushes code or creates a pull request in GitHub.
2. GitHub Actions checks out the source code and installs dependencies.
3. The build process prepares the application and AI service components.
4. Automated unit and integration tests are executed.
5. API and regression tests run for critical business and quality workflows.
6. Sonar Qube performs code-quality analysis and enforces the quality gate.
7. Security and dependency scans check for vulnerabilities and exposed secrets.
8. If all checks pass, a versioned Docker image is built and pushed to the registry.
9. The image is deployed to Development and smoke-tested.
10. The same image is promoted to Staging and validated in a production-like environment.
11. After approval, the version is deployed to Production with health monitoring.
12. If health checks or critical monitoring alerts fail, the previous stable version is restored.

9. Conclusion

The proposed CI/CD design provides a complete and controlled delivery process for the AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform. It covers source management, automated build, multi-level testing, code-quality analysis, security checks, container packaging, staged deployment, monitoring, notification, and rollback. This approach improves software reliability and traceability while reducing the risk of deploying untested or vulnerable changes into a quality-critical pharmaceutical manufacturing environment.

10. Source Code Management and Branching Strategy

Protected branches can prevent direct unreviewed changes to the stable branch. Required status checks, code review, and successful CI runs provide an additional layer of control before integration.

A clear naming convention for branches and tags improves traceability. For example, feature branches can correspond to individual enhancements while release tags identify the exact source state associated with a packaged and deployed version.

The project repository is maintained using Git-based version control. Developers work in isolated feature branches so that incomplete changes do not directly affect the stable codebase.

Each feature branch is linked to a specific requirement, defect correction, enhancement, or quality improvement. Pull requests provide a controlled mechanism for reviewing code before integration.

The main branch represents the approved integration baseline. Only changes that pass required automated checks and review conditions are merged. Release versions are tagged so that a deployed build can always be traced back to its source code.

This strategy improves accountability, reproducibility, and rollback capability because every release has a known commit and version history.

11. Continuous Integration Build Process

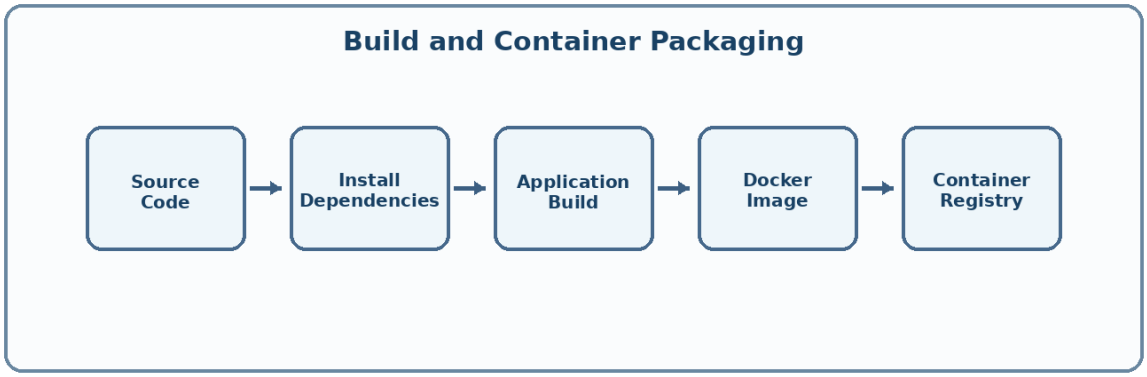


Figure: Build and container packaging process

Build reproducibility is important for reliable delivery. Dependency versions should be controlled so that a future build does not unexpectedly use a different library version from the one that was previously tested.

Build logs should be retained with the pipeline result. These logs provide evidence of the build environment, executed steps, warnings, and errors.

Continuous Integration begins whenever a developer pushes code or updates a pull request. The pipeline checks out the correct revision and installs the dependencies required by the application.

The build process validates that the web application, backend services, AI/ML components, configuration files, and supporting modules can be assembled successfully.

A failed build immediately stops the pipeline. This prevents invalid source code or missing dependencies from moving to testing and deployment stages.

Successful builds create a consistent basis for subsequent tests. The same approved build output is used for packaging so that later environments do not use a different unverified version.

12. Unit Testing Strategy

Unit tests should be fast enough to execute frequently. Fast feedback encourages developers to identify defects immediately after a code change rather than waiting until a later integration environment.

Code coverage can be used as a supporting metric, but coverage alone is not treated as proof of correctness. The quality of test cases and the behaviour they validate are also important.

Unit tests focus on individual software components. Important examples include data validation functions, authentication routines, quality-score calculations, anomaly-processing interfaces, report generation functions, and notification logic.

Tests include normal inputs, boundary values, invalid inputs, and error conditions. This helps identify defects at the earliest and least expensive stage of the delivery process.

Unit tests are executed automatically for every relevant code change. A minimum pass requirement can be configured as a pipeline quality gate.

The results are stored with the pipeline run so that developers can identify failed tests and correct the responsible code before the release proceeds.

13. Integration and API Testing

Integration tests can use controlled test databases and mock services when external dependencies are unavailable. The test environment should be reset to a known state so that results are repeatable.

API contracts should remain stable or be versioned carefully. Contract checks can detect unintended changes that would break other services or client applications.

Integration testing verifies that independently developed components work correctly when connected together. The platform requires reliable interaction between user interfaces, backend APIs, databases, AI/ML services, and alerting modules.

API tests verify valid requests, invalid requests, authentication behaviour, response formats, error handling, and critical quality-assurance workflows.

Representative test cases simulate the flow of manufacturing or batch information through validation, analysis, anomaly detection, and report generation.

The pipeline blocks promotion when critical integration or API tests fail because a component can pass unit tests while still failing when connected to other services.

14. Regression and Smoke Testing

Regression suites can be divided into fast and full groups. Fast regression tests run on every change, while larger suites can run before promotion to Staging or Production.

Smoke tests focus on release readiness. Their purpose is not to test every feature, but to quickly confirm that the deployed application is usable at a basic operational level.

Regression testing protects existing functionality when new features or fixes are introduced. Previously approved workflows are re-executed to ensure that a change does not unintentionally damage another feature.

Smoke testing is performed after deployment to verify that the most important services start correctly and that essential functions are available.

Typical smoke checks include application availability, login access, API responsiveness, database connectivity, AI-service communication, and alert/report generation.

These automated checks provide fast feedback after deployment and help detect configuration or environment problems before users depend on the new release.

15. Code Quality Analysis

Code-quality rules should be agreed by the project team and applied consistently. The goal is to reduce technical debt, simplify maintenance, and prevent known defect patterns from becoming part of the stable codebase.

Quality analysis reports can be reviewed together with test and security results during release evaluation. This creates a broader view of release readiness than any single metric.

Code-quality analysis is integrated into the pipeline through a quality analysis tool such as Sonar Qube. The objective is to identify maintainability problems before deployment.

The analysis can report bugs, duplicated code, code smells, reliability concerns, and other indicators that affect long-term maintainability.

A quality gate defines the minimum acceptable condition for the project. If configured thresholds are not met, the pipeline stops before packaging or promotion.

This stage is important because a successful build alone does not guarantee that the software is maintainable, readable, or sufficiently reliable for continued development.

16. Security Validation in the Pipeline

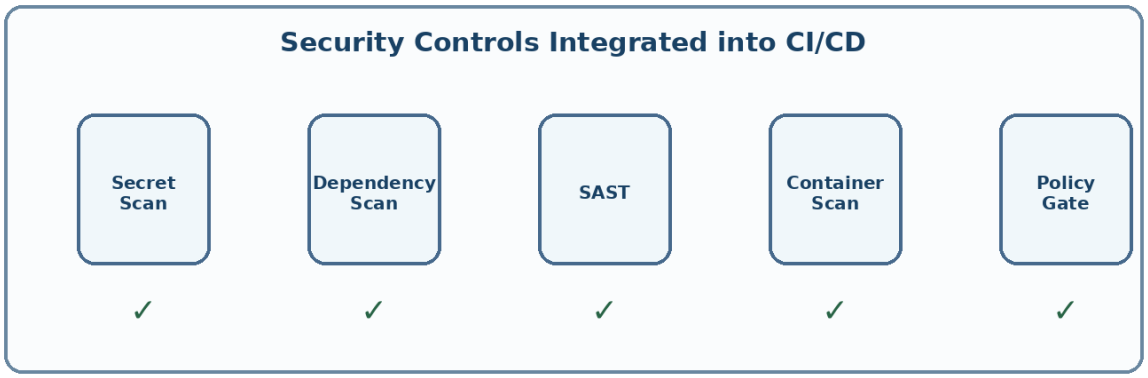


Figure: Security controls integrated into the pipeline

Security tools require regular updates so that scans recognise newly published vulnerability information. The pipeline should also record the scanned dependency and image versions for later review.

Access to deployment credentials should follow the principle of least privilege. A CI/CD job should receive only the permissions required for its defined stage.

Security checks are performed automatically rather than being postponed until the end of development. Source code and dependencies are checked for known vulnerabilities and insecure components.

Secret scanning helps prevent passwords, tokens, API keys, and other credentials from being committed into the repository.

Container images are also scanned before deployment. Critical or high-risk findings can be configured to block the release until they are reviewed and corrected.

Secure configuration is maintained through protected environment variables or secret-management facilities. Sensitive values are injected during execution instead of being stored directly in application code.

17. Container Packaging and Artifact Management

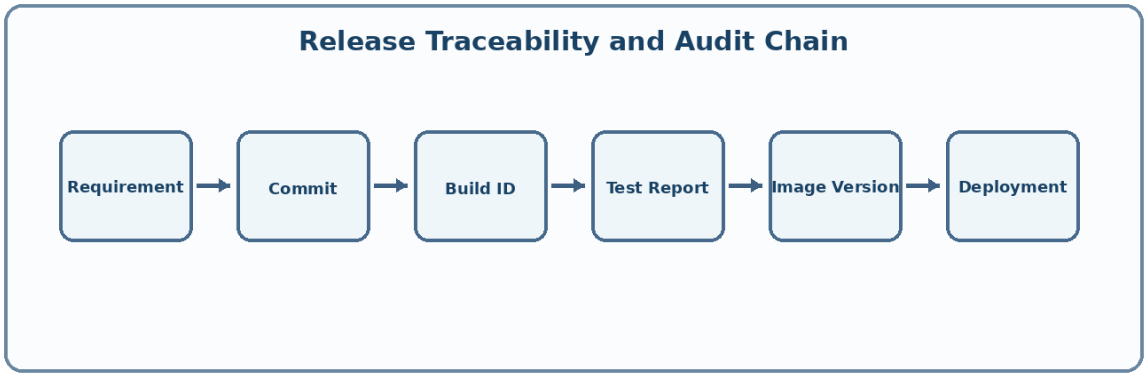


Figure: Release traceability from requirement to deployment

Container tags should avoid ambiguity. A unique build identifier or immutable digest makes it easier to confirm exactly which artifact was promoted.

Old artifacts can be retained according to project policy so that a known stable version remains available when rollback or investigation is required.

After the build, tests, quality analysis, and security checks succeed, the approved application is packaged as a versioned Docker image.

The image contains the software and its required runtime dependencies, creating consistent execution across Development, Staging, and Production environments.

Each image is assigned a traceable version or release tag. The registry stores approved artifacts so that the exact deployed version can be retrieved when required.

Artifact versioning also supports rollback because the previously stable image remains identifiable and available for controlled restoration.

18. Development and Staging Deployment

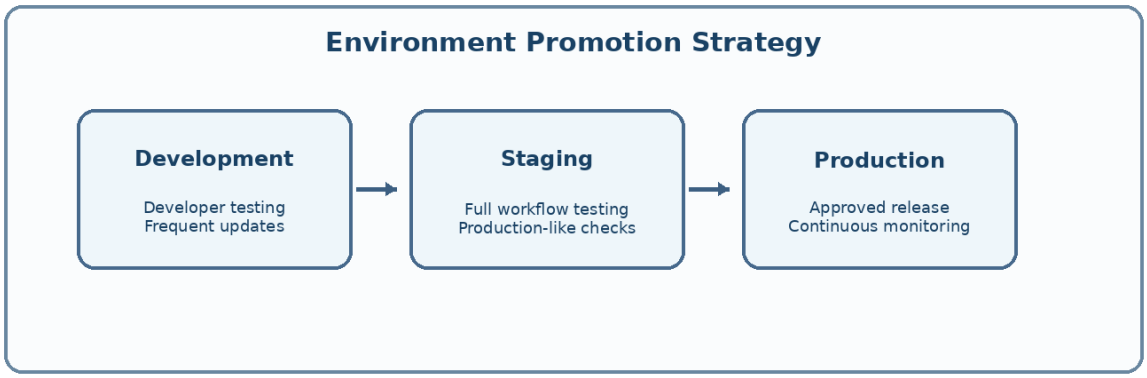


Figure: Controlled promotion across deployment environments

Staging should include representative configuration and integration conditions while protecting sensitive production information. Test data can be used when real operational data is not appropriate for the environment.

Deployment automation should be idempotent where possible, meaning that repeating a deployment command produces the intended environment state without creating uncontrolled duplication.

The Development environment provides rapid feedback after successful CI validation. It is used to verify the integrated application in an environment separate from individual developer machines.

After Development checks succeed, the same approved release artifact is promoted to Staging. Staging is configured to represent production as closely as practical.

Broader integration, regression, API, workflow, and smoke tests are performed before Production approval.

Using the same versioned artifact across environments reduces the risk that a different build reaches Production without having been tested.

19. Production Deployment and Monitoring

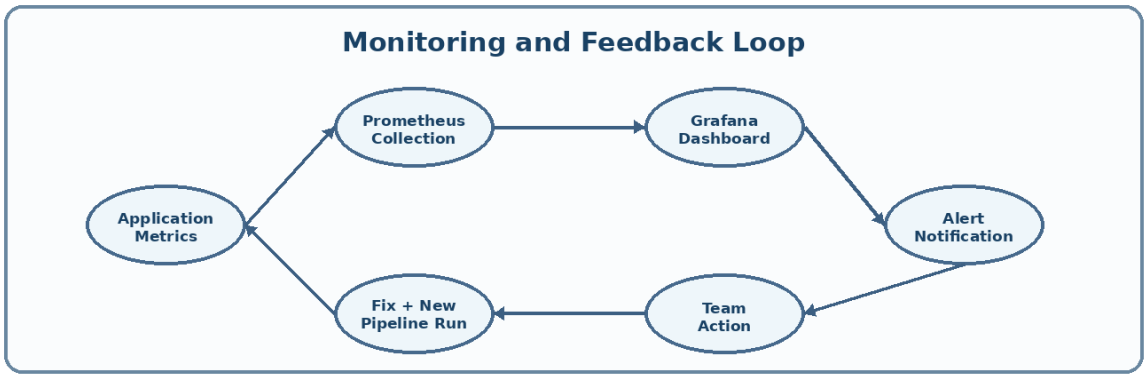


Figure: Monitoring and continuous feedback loop

Production deployment can use controlled strategies such as rolling updates or blue-green deployment when the infrastructure supports them. The chosen method should minimize disruption and make recovery straightforward.

Release monitoring is particularly important immediately after deployment. A defined observation period allows the team to confirm that error rates and health indicators remain within acceptable limits.

Production deployment occurs only after the required CI/CD gates and approvals have been completed. The release is traceable through its source commit, build record, test results, and image version.

Health checks are used immediately after deployment to confirm that critical services remain operational.

Monitoring collects metrics such as service availability, response behaviour, resource usage, error rates, and application health indicators.

Dashboards and alerts provide operational visibility. When a critical condition occurs, responsible team members are notified so that investigation can begin quickly.

20. Notification, Failure Handling and Rollback

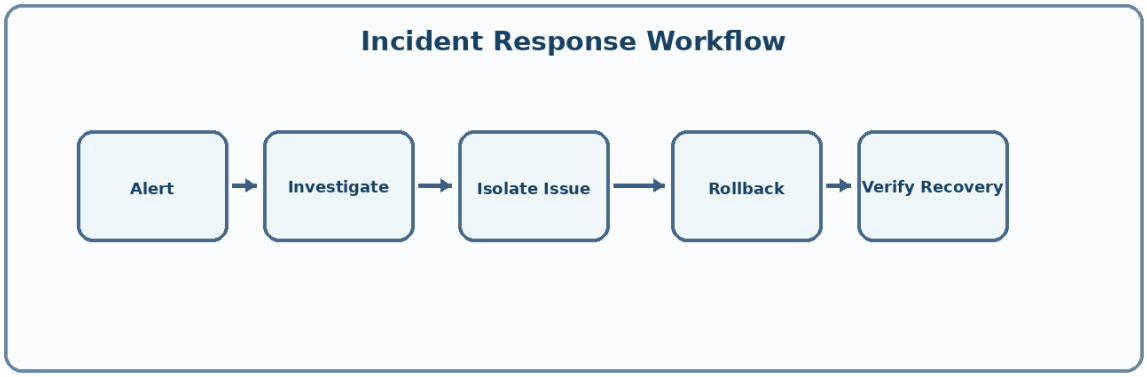


Figure: Incident response and recovery workflow

Notifications should include useful context such as the pipeline stage, release version, affected environment, failure summary, and a reference to detailed logs.

After a rollback, the failed version should not be automatically promoted again. The underlying cause must be investigated and corrected before a new release enters the pipeline.

Every important pipeline outcome should be visible to the responsible team. Notifications can be generated for build failures, failed tests, quality-gate failures, security findings, deployment success, and production alerts.

The pipeline follows a fail-fast approach: when a critical stage fails, later stages are not executed. This prevents an invalid release from progressing further.

Rollback is based on the previously approved stable version. If the new deployment fails health checks or produces a critical operational issue, the stable version can be restored.

The rollback event is recorded for traceability, and the failed release is investigated before another deployment attempt is made.

21. Final CI/CD Design Summary

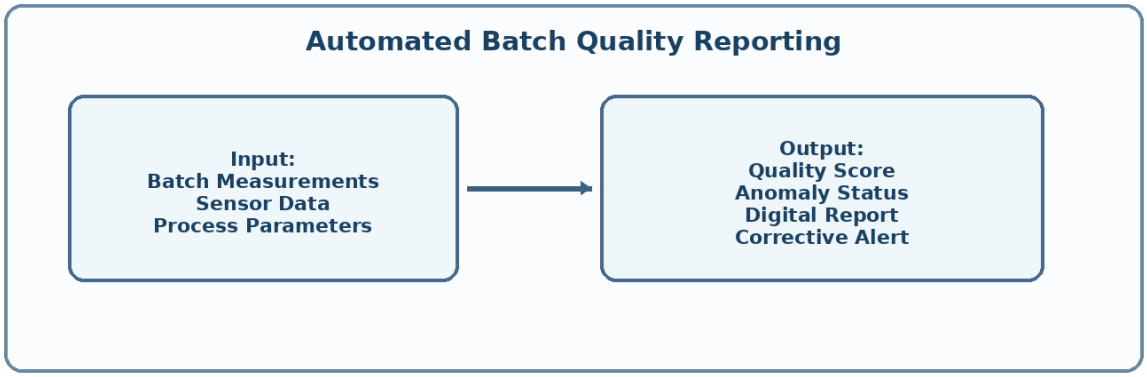


Figure: Automated generation of quality outcomes and reports

The design is based on the principle that software quality is built continuously rather than checked only at the end. Automated feedback at every stage helps identify defects and risks close to the change that introduced them.

By combining testing, quality analysis, security validation, controlled promotion, monitoring, and rollback, the pipeline provides a practical DevOps framework for the proposed software engineering project.

The complete design connects source code management, automated build, testing, code quality, security validation, container packaging, environment promotion, monitoring, notification, and rollback into one controlled workflow.

The strongest feature of the design is the use of quality gates at multiple stages. A change cannot progress simply because it compiles; it must also satisfy functional, integration, quality, and security requirements.

For the AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform, this approach supports reliable and repeatable software delivery while maintaining traceability of changes and releases.

The proposed pipeline therefore satisfies the assessment focus on robust testing, quality assurance, DevOps practices, security, reliability, deployment strategy, documentation, and rollback mechanisms.

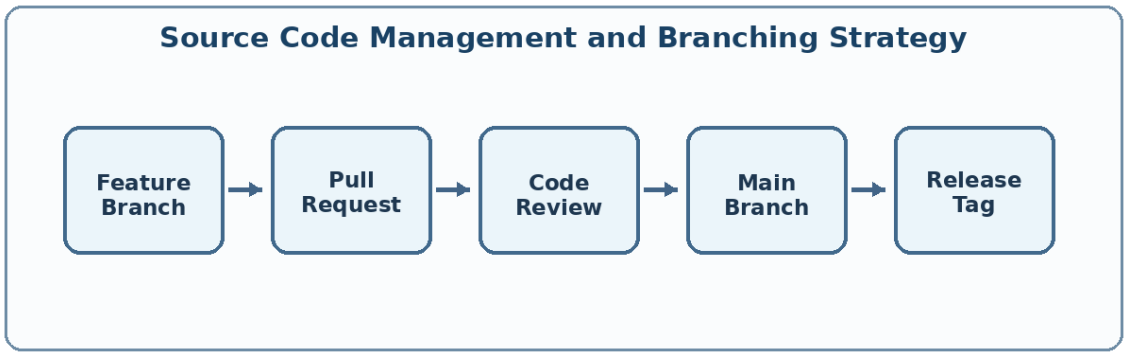


Figure: Controlled branching and release flow

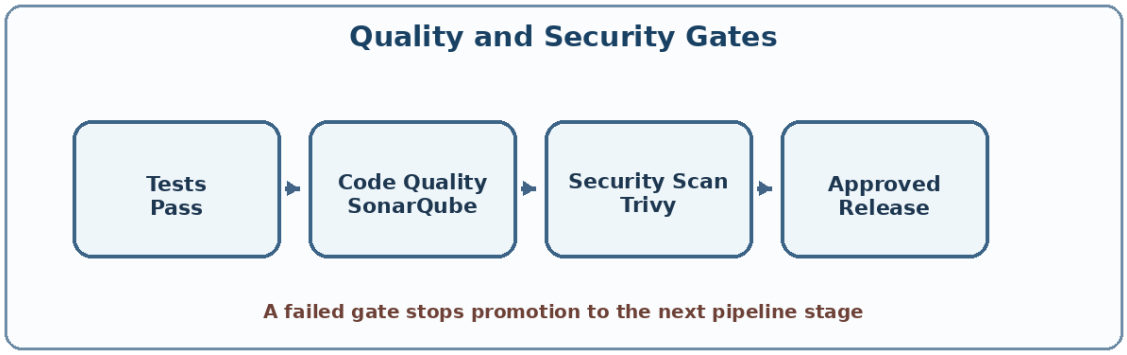


Figure: Quality and security gates before release packaging

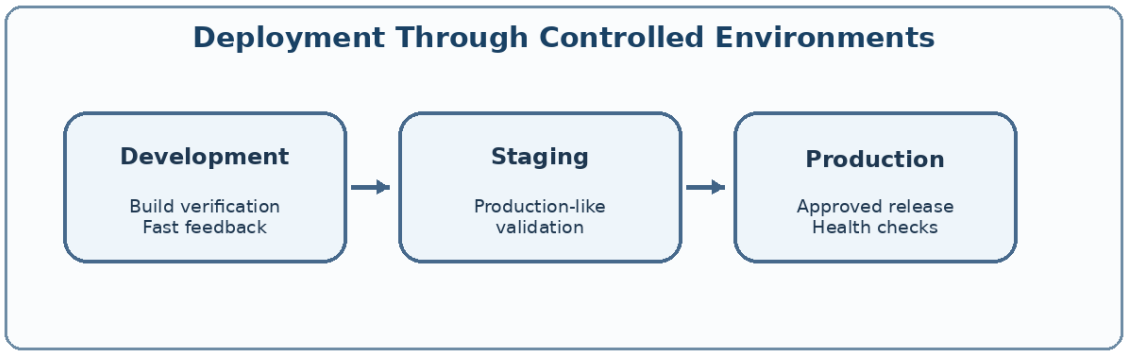


Figure: Promotion of the same approved release across environments

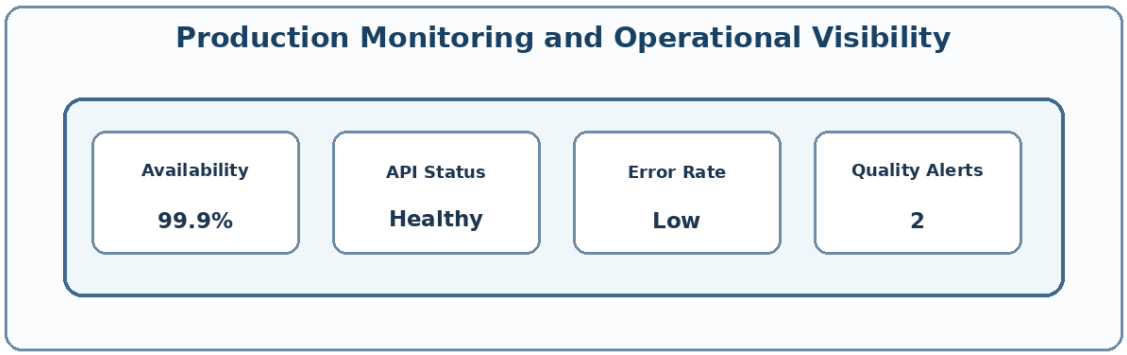


Figure: Example production monitoring indicators

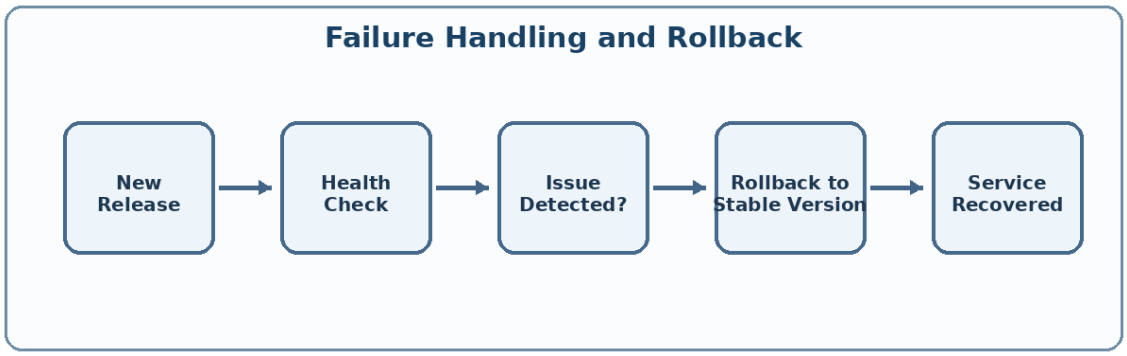


Figure: Health-check failure and rollback to a stable release